

Experiment 8: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR).

Theory:

1. Dataset Source

Dataset used: **MNIST Handwritten Digit Dataset**

Source: TensorFlow Datasets

Link: <https://www.tensorflow.org/datasets/catalog/mnist>

2. Dataset Description

The MNIST dataset is a widely used benchmark dataset for image classification tasks.

Features:

- Each image is a **28 × 28 grayscale image**
- Pixel values range from **0 to 255**

Target Variable:

- Digit label (0 to 9)

Dataset Size:

- Training samples: **60,000 images**
- Testing samples: **10,000 images**

Characteristics:

- Simple and clean dataset
- Suitable for beginners in deep learning
- Used for handwritten digit recognition

3. Mathematical Formulation of CNN

Convolution Operation:

$$\text{Feature Map} = \text{Input} * \text{Kernel}$$

Neuron Output:

$$Z = \sum (w_i x_i) + b$$

$$A = f(Z)$$

Activation Function (ReLU):

$$f(x) = \max(0, x)$$

Softmax Function (Output Layer):

$$P(y_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

Loss Function (Categorical Crossentropy):

$$\text{Loss} = - \sum y \log(\hat{y})$$

4. Algorithm Limitations

- Requires **large computational resources**
- Needs **large dataset** for better performance
- Training time is high
- Difficult to interpret (**black-box model**)
- Sensitive to hyperparameters
- May overfit if not properly regularized

5. Methodology / Workflow

Steps Involved:

1. **Dataset Loading**
Load MNIST dataset
2. **Data Preprocessing**
 - Normalize pixel values (0–1)
 - Reshape images for CNN
3. **Model Building**

- Add convolutional layers
- Add pooling layers
- Add fully connected layers
- 4. **Model Compilation**
 - Define optimizer (Adam)
 - Define loss function
- 5. **Model Training**
 - Train model using epochs
- 6. **Model Evaluation**
 - Calculate accuracy and loss
- 7. **Visualization**
 - Plot accuracy and loss graphs

6. Performance Analysis

- Model achieved accuracy of approximately **99%**
- **Accuracy graph** shows increasing trend
- **Loss graph** shows decreasing trend

Interpretation:

- Model learns quickly in early epochs
- High accuracy indicates **excellent classification performance**
- Small gap between training and validation → **no overfitting**

7. Hyperparameter Tuning

Hyperparameters Used:

- Epochs: 5
- Batch Size: 32
- Filters: 32, 64
- Kernel Size: (3,3)
- Optimizer: Adam

Impact:

- Increasing epochs improved accuracy
- Proper filter size improved feature extraction
- Adam optimizer ensured faster convergence

Code:

```
# Step 1: Import Libraries
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Step 2: Load Dataset (MNIST)
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# Step 3: Preprocessing
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

# Step 4: Build CNN Model
model = models.Sequential()

model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

# Step 5: Compile Model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Step 6: Train Model
history = model.fit(X_train, y_train, epochs=5,
                   validation_data=(X_test, y_test))

# Step 7: Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)

# Step 8: Plot Accuracy Graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
```

```
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Test'])

plt.show()

# Step 9: Plot Loss Graph
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train', 'Test'])

plt.show()
```

Output:

Step 6: Train Model

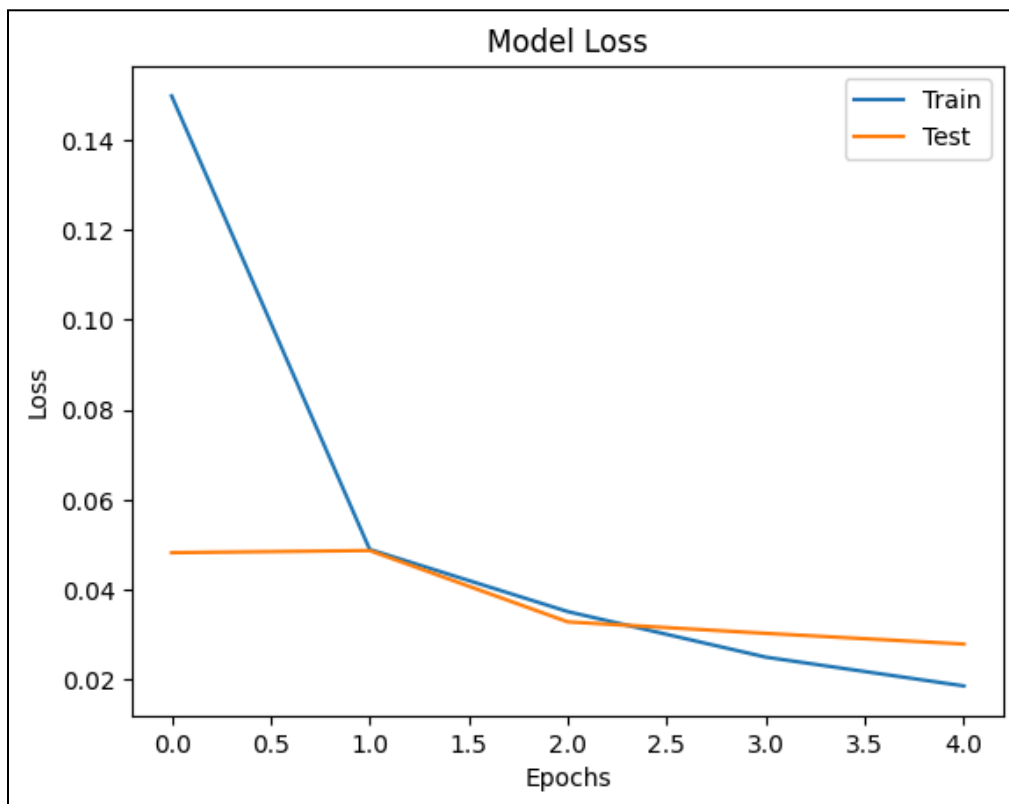
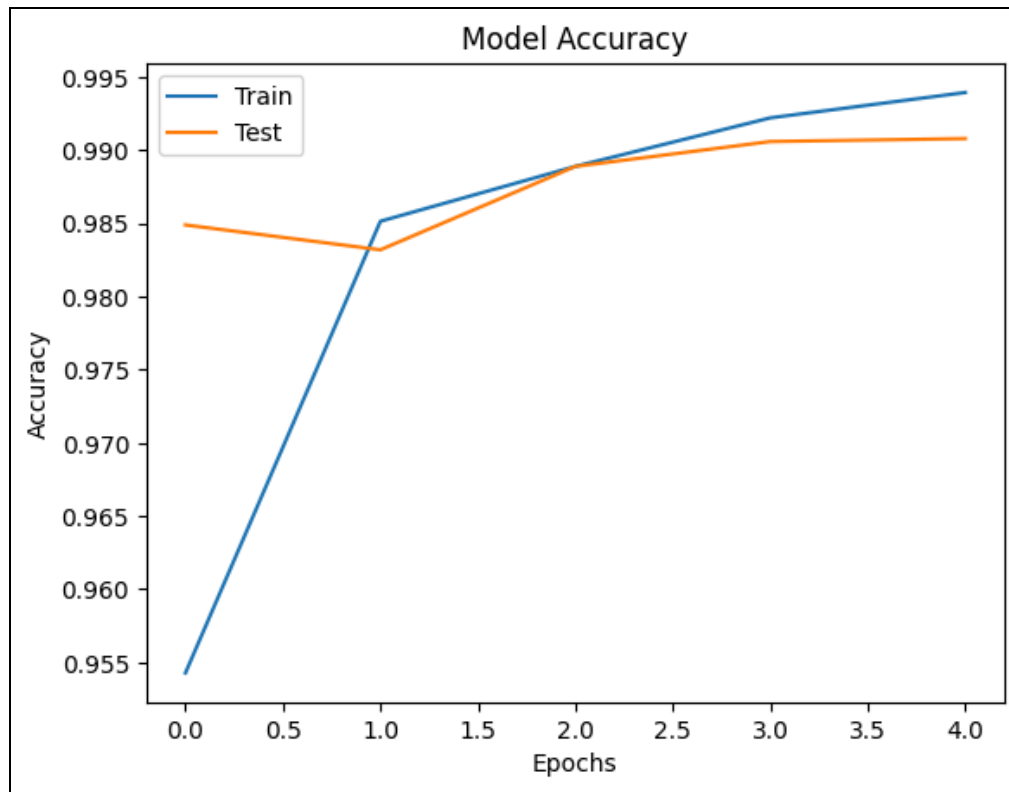
```
history = model.fit(X_train, y_train, epochs=5,
                    validation_data=(X_test, y_test))
```

```
Epoch 1/5
1875/1875 ————— 67s 34ms/step - accuracy: 0.9543 - loss: 0.1498 - val_accuracy: 0.9849 - val_loss: 0.0482
Epoch 2/5
1875/1875 ————— 66s 35ms/step - accuracy: 0.9851 - loss: 0.0489 - val_accuracy: 0.9832 - val_loss: 0.0487
Epoch 3/5
1875/1875 ————— 63s 34ms/step - accuracy: 0.9889 - loss: 0.0351 - val_accuracy: 0.9889 - val_loss: 0.0328
Epoch 4/5
1875/1875 ————— 76s 30ms/step - accuracy: 0.9922 - loss: 0.0249 - val_accuracy: 0.9906 - val_loss: 0.0302
Epoch 5/5
1875/1875 ————— 55s 29ms/step - accuracy: 0.9940 - loss: 0.0185 - val_accuracy: 0.9908 - val_loss: 0.0278
```

Step 7: Evaluate Model

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)
```

```
313/313 ————— 3s 8ms/step - accuracy: 0.9908 - loss: 0.0278
Accuracy: 0.9908000230789185
```



Graph Explanation

1. Accuracy Graph

- **Training Accuracy** increases from ~95% to ~99.4%
- **Validation (Test) Accuracy** increases from ~98.5% to ~99.1%

Observation:

- Both curves increase rapidly in early epochs
- After epoch 2, accuracy **stabilizes near 99%**
- Training accuracy is slightly higher than validation accuracy

Interpretation:

- Model is **learning very quickly and effectively**
- High accuracy indicates **excellent performance**
- Small gap between curves → **no significant overfitting**

2. Loss Graph

- **Training Loss** decreases from ~0.15 to ~0.02
- **Validation Loss** decreases from ~0.048 to ~0.028

Observation:

- Sharp drop in loss in initial epochs
- Both curves gradually stabilize
- Training loss is slightly lower than validation loss

Interpretation:

- Decreasing loss shows **improving predictions**
- Close curves indicate **stable and well-trained model**
- No increase in validation loss → **no overfitting**

Conclusion:

- The **Convolutional Neural Network (CNN)** was successfully designed and trained on the image dataset
- The model achieved a very high accuracy of approximately **99%**, indicating **excellent performance** in image classification.
- The **accuracy graph** showed a rapid increase, while the **loss graph** showed a continuous decrease, confirming effective learning.
- The close alignment of **training and validation curves** indicates **no significant overfitting** and good generalization.
- Thus, CNN proved to be a **powerful and efficient deep learning model** for image-based tasks such as **digit recognition, object detection, and image classification**.